NeRF Architectures under Sparse-View Conditions: A Systematic Benchmarking Methodology

Felipe G. Silva*, Leandro A. Neves*, Guilherme F. Roberto*, Thiago Pozati*, Domingos Oliveira†,‡, Adriano B. Silva‡, Thaína A. A. Tosta§ and Marcelo Z. do Nascimento‡
*Department of Computer Science and Statistics (DCCE). São Paulo State University (UNESP), São José do Rio Preto, SP, Brazil. Email: felipe.gomes-silva@unesp.br
†Federal Institute of Education, Science and Technology of São Paulo (IFSP), São Paulo, SP, Brazil.
‡Faculty of Computer Science (FACOM), Federal University of Uberlândia (UFU), Uberlândia, MG, Brazil
§Department of Computing and Mathematics, University of São Paulo (USP), Ribeirão Preto, SP, Brazil

Abstract*—Novel view synthesis from few input images ($N \leq 10$) involves a direct trade-off between learned geometric priors, optimization stability, and rendering speed. This work presents a reproducible benchmarking methodology that compares generalizable implicit architectures (PixelNeRF and GNT) under five fine-tuning strategies (Full Fine-Tuning, PFT-Decoder, PFT-Backbone, BitFit, and PEFT-LoRA) against implicit and explicit per-scene representations (Instant-NGP, Nerfacto, and 3D Gaussian Splatting via Splatfacto) on the LLFF dataset. Quantitative and qualitative evaluations were conducted using PSNR, SSIM, LPIPS, training time, rendering throughput (FPS), and dynamic VRAM consumption. In the experimental results, per-scene optimization achieved the highest performance under 10 input views: Nerfacto reached 27.11 dB PSNR, 0.875 SSIM, and 0.171 LPIPS, while Splatfacto rendered novel views at 200.7 FPS using 0.10 GB of VRAM (24.80 dB PSNR). Under three input views ($N = 3$), modular parameter adaptation prevented representational collapse in pre-trained models: GNT adapted with PFT-Decoder (15.07 dB) and PEFT-LoRA ($r = 4$, 14.57 dB) matched Full Fine-Tuning accuracy while reducing adaptation time by 16.3% ($\sim$5 min) and outperforming Instant-NGP (14.06 dB), although limited by volumetric ray-marching latency (≤ 0.015 FPS). The custom Nerfstudio wrappers and benchmarking scripts were publicly released to support reproducible evaluation.

I. Introduction

Neural Radiance Fields (NeRFs) and explicit point-based representations such as 3D Gaussian Splatting (3DGS) have become standard approaches for novel view synthesis [1]–[3]. However, most neural rendering methods require dense multi-view supervision and long optimization times, which limits their application in acquisition-constrained settings such as mobile scanning, robotic navigation, and 3D biomedical imaging [4], [5].

Recent architectures have addressed these limitations through two distinct approaches. Per-scene methods reduce training and inference costs using multiresolution hash grids, as in Instant-NGP [6] and Nerfacto [3], or explicit tile-based Gaussian rasterization, as in Splatfacto [2]. In parallel, generalizable architectures learn multi-view priors across large datasets to synthesize novel views from few source images, as demonstrated by PixelNeRF [7] and the Generalizable NeRF Transformer (GNT) [8]. Although generalizable models support feed-forward inference and short fine-tuning, their behavior under parameter-efficient fine-tuning (PEFT) and their computational trade-offs relative to implicit and explicit per-scene baselines have largely been evaluated under separate experimental protocols [9].

This work proposes a standardized benchmarking methodology to evaluate generalizable and per-scene neural rendering architectures under sparse-view conditions ($N \leq 10$ on the LLFF dataset). Reconstruction fidelity (PSNR, SSIM [10], and LPIPS [11]) and computational performance (training time, rendering throughput in FPS, and peak dynamic VRAM) were jointly measured. The main contributions of this study are:

- A unified evaluation protocol comparing per-scene models (Instant-NGP, Nerfacto, and Splatfacto/3DGS) and pre-trained generalizable models (PixelNeRF and GNT) across zero-shot and stratified K-fold fine-tuning regimes.
- Publicly released Nerfstudio integration wrappers (`nerfstudio-gnt` and `nerfstudio-pixelnerf`) implementing five adaptation strategies (Full-FT, PFT-Decoder, PFT-Backbone, BitFit, and PEFT-LoRA) with in-place weight merging for inference.

II. Theoretical Background

A. Neural Radiance Fields and Volumetric Rendering

Neural Radiance Fields (NeRFs) represent continuous 3D scene geometry and view-dependent appearance by mapping spatial coordinates $\mathbf{x} = (x, y, z)$ and viewing directions $\mathbf{d} = (\theta, \phi)$ to volume density $\sigma(\mathbf{x}) \in \mathbb{R}_{\geq 0}$ and directional radiance $\mathbf{c}(\mathbf{x}, \mathbf{d}) \in [0, 1]^3$ using a Multilayer Perceptron (MLP) F_θ [1]. Sinusoidal positional encoding $\gamma(\cdot)$ maps the inputs into a higher-dimensional Fourier space $\mathbf{v} = \gamma(\mathbf{x}, \mathbf{d})$ so that $F_\theta(\mathbf{v}) \mapsto (\sigma, \mathbf{c})$ represents high-frequency geometric and textural details. The color observed along a camera ray $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$ within bounds $[t_n, t_f]$ is computed via the volumetric rendering integral [1]:

$$\hat{C}(\mathbf{r}) = \int_{t_n}^{t_f} T(t)\, \sigma(\mathbf{r}(t))\, \mathbf{c}(\mathbf{r}(t), \mathbf{d})\, dt \tag{1}$$

where $T(t) = \exp\Big(-\int_{t_n}^{t} \sigma(\mathbf{r}(s))\, ds\Big)$ denotes the accumulated transmittance, which is numerically estimated through stratified quadrature sampling.

B. Explicit Primitives, Hybrid Grids, and Generalization

Evaluating coordinate-based MLPs across thousands of ray samples introduces high computational costs during training and rendering. Hybrid per-scene representations mitigate this cost by storing trainable features in spatial data structures. Instant-NGP [6] indexes feature vectors across multiresolution hash tables, reducing training times from hours to minutes. Nerfacto [3] combines hash encodings with proposal network sampling, spherical harmonics, and appearance embeddings.

In contrast to implicit ray-marching, 3D Gaussian Splatting (3DGS) [2] (implemented in Nerfstudio as Splatfacto) represents a scene explicitly as a set of anisotropic 3D Gaussian primitives $\{\mathcal{G}_i\}_{i=1}^{M}$. Each primitive is defined by a 3D center $\boldsymbol{\mu}_i \in \mathbb{R}^3$, an opacity value $\alpha_i \in [0,1]$, spherical harmonics coefficients $\mathbf{c}_i$, and a positive semi-definite 3D covariance matrix factorized into scaling $\mathbf{S}_i$ and rotation $\mathbf{R}_i$ matrices:

$$\boldsymbol{\Sigma}_i = \mathbf{R}_i \mathbf{S}_i \mathbf{S}_i^\top \mathbf{R}_i^\top, \quad \boldsymbol{\Sigma}_i' = (\mathbf{J}\mathbf{W}\boldsymbol{\Sigma}_i\mathbf{W}^\top\mathbf{J}^\top)_{1:2,1:2} \tag{2}$$

where $\mathbf{W}$ is the viewing transformation and $\mathbf{J}$ is the affine Jacobian of the projective transformation. Given the 2D Gaussian response $\alpha_i' = \alpha_i \exp\Big(-\frac{1}{2}(\mathbf{p}-\boldsymbol{\mu}_i')^\top {\boldsymbol{\Sigma}_i'}^{-1}(\mathbf{p}-\boldsymbol{\mu}_i')\Big)$ at pixel $\mathbf{p}$, a tile-based differentiable rasterizer sorts the primitives by depth and blends pixel colors from front to back:

$$\hat{C}(\mathbf{p}) = \sum_{i\in\mathcal{N}} \mathbf{c}_i \alpha_i' \prod_{j=1}^{i-1}(1-\alpha_j') \tag{3}$$

Although per-scene optimization achieves high fidelity when training views are dense, sparse-view settings ($N \leq 10$) leave the 3D reconstruction problem underconstrained, which often produces floating artifacts and inaccurate depth estimates [5], [12], [13]. Generalizable NeRFs address view scarcity by learning multi-view priors across large scene collections [7], [8], [14], [15]. PixelNeRF [7] projects 3D query points onto source image planes and conditions a coordinate MLP on extracted ResNet features. The Generalizable NeRF Transformer (GNT) [8] replaces the convolutional backbone and MLP renderer with a Vision Transformer (ViT) encoder and an epipolar cross-attention decoder that aggregates multi-view features along epipolar lines.

C. Fine-Tuning Strategies and Parameter-Efficient Regimes

When pre-trained generalizable models are applied directly in a zero-shot (ZS) regime to novel scenes, differences in camera intrinsics, lighting, and textures often cause spatial blurring under few input views ($N \leq 10$) [7], [16]. Fine-tuning (FT) adapts the network weights to the target scene by minimizing the photometric reconstruction loss over sampled rays $\mathcal{R}$:

$$\mathcal{L}_{\text{photo}} = \frac{1}{|\mathcal{R}|}\sum_{\mathbf{r}\in\mathcal{R}} \|\hat{C}(\mathbf{r}) - C(\mathbf{r})\|_2^2 \tag{4}$$

However, unconstrained optimization over few rays can overfit to the training views, degrading the pre-trained priors and introducing density floaters near the camera planes [5], [17]. Following the adaptation taxonomy proposed by Gupta et al. [18], five fine-tuning strategies were defined for generalizable neural rendering:

1) Full Fine-Tuning (Full-FT): All network parameters $\Theta = \{W, b\}$ across the 2D feature backbone and the 3D rendering network are updated simultaneously. Despite allowing full weight adjustment, Full-FT requires storing optimizer states for all layers and increases the risk of overfitting when few views are available [5], [18].

2) Partial Fine-Tuning: Decoder (PFT-Decoder): Gradient updates are restricted to the volumetric rendering head (the epipolar cross-attention transformer in GNT or the radiance MLP in PixelNeRF), while the 2D spatial encoder remains frozen. Keeping the visual encoder fixed preserves the pre-trained 2D representations and directs adaptation toward multi-view radiance aggregation [18].

3) Partial Fine-Tuning: Backbone (PFT-Backbone): PFT-Backbone updates only the 2D feature extractor (the ResNet in PixelNeRF or the patch embedder in GNT) while keeping the volumetric rendering head frozen. This configuration adjusts low-level perceptual feature maps to the target textures and illumination without altering the pre-trained ray integration layers.

4) Selective PEFT: BitFit: BitFit freezes all weight matrices $W \in \mathbb{R}^{d_{\text{out}}\times d_{\text{in}}}$ and optimizes only the additive bias vectors according to $y = Wx + (b + \Delta b)$ [19]. Updating less than 0.15% of the total parameters acts as a structural constraint that shifts activation thresholds while preserving the pre-trained weights [18].

5) Additive PEFT: Low-Rank Adaptation (PEFT-LoRA): LoRA freezes the pre-trained weight matrices $W_0 \in \mathbb{R}^{d_{\text{out}}\times d_{\text{in}}}$ and represents weight updates through a low-rank decomposition [20]:

$$W' = W_0 + \Delta W = W_0 + \frac{\alpha}{r} BA \tag{5}$$

where $B \in \mathbb{R}^{d_{\text{out}}\times r}$ (initialized to zero) and $A \in \mathbb{R}^{r\times d_{\text{in}}}$ (initialized from a Gaussian distribution) have rank $r \ll \min(d_{\text{in}}, d_{\text{out}})$, scaled by α. Constraining the update to a low-rank subspace reduces optimizer memory and regularizes fine-tuning under limited supervision [18].

III. Materials and Methods

This section describes the experimental methodology used to evaluate computational efficiency and reconstruction fidelity across neural rendering architectures under sparse-view conditions. Figure 1 illustrates the three stages of the pipeline: (i) dataset preparation and sparse-view sampling; (ii) model configuration across generalizable adaptation and per-scene training regimes; and (iii) quantitative and qualitative evaluation on held-out views.

A. Dataset & Splits

All experiments were conducted on the Local Light Field Fusion (LLFF) dataset [16], which contains 8 forward-facing real-world scenes with COLMAP-calibrated camera poses and depth bounds, downscaled by a factor of 8 (504×378 pixels). Following the few-shot evaluation protocol of Niemeyer et al.

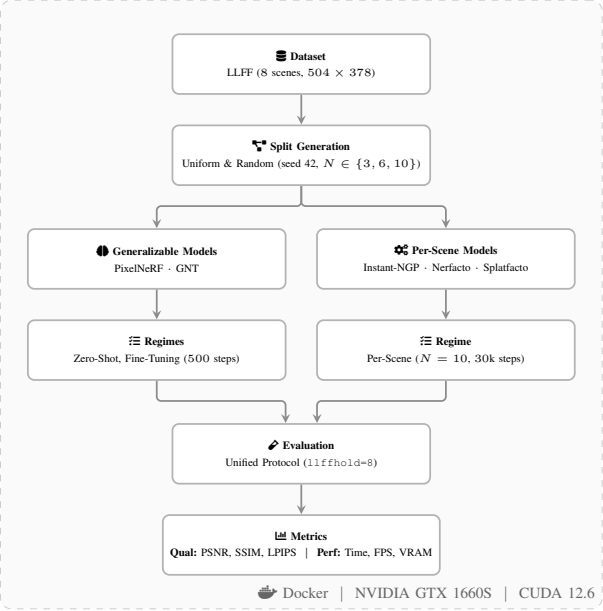


Fig. 1. Proposed benchmarking pipeline. COLMAP-calibrated sparse splits ($N \in \{3, 6, 10\}$) were evaluated across two architectural branches inside an isolated Docker environment: generalizable models under Zero-Shot and five fine-tuning strategies, and per-scene baselines trained to convergence, assessed via visual fidelity (PSNR, SSIM, LPIPS) and computational metrics (Time, FPS, VRAM).

TABLE I
COMPARISON BETWEEN UNIFORM AND RANDOM TRAINING VIEW SELECTION UNDER $N = 10$ VIEWS ON THE *fern* SCENE. UNIFORM SAMPLING ALONG THE CAMERA TRAJECTORY IMPROVED RECONSTRUCTION METRICS ACROSS BOTH GENERALIZABLE AND PER-SCENE MODELS.

Model	Split	PSNR ↑	SSIM ↑	LPIPS ↓	ΔPSNR
PixelNeRF	Uniform	6.98	0.047	0.846	**+0.54**
	Random	6.44	0.042	0.840	
GNT	Uniform	10.34	0.179	0.738	+0.15
	Random	10.19	0.184	0.742	
Instant-NGP	Uniform	**13.57**	**0.231**	**0.627**	+0.06
	Random	13.51	0.229	0.630	
Nerfacto	Uniform	12.04	0.227	0.809	+0.11
	Random	11.93	0.226	0.865	

[5], every 8th image (`llffhold=8`) was reserved for testing, while the training subsets were restricted to $N \in \{3, 6, 10\}$ views. Two view selection criteria were compared on the *fern* scene ($N = 10$, Table I): (i) *Uniform*, where training views were sampled by linear interpolation over the dataset indices along the camera trajectory; and (ii) *Random*, where views were drawn using a fixed seed (`seed=42`). Uniform sampling yielded higher PSNR across all tested models (up to $+0.54$ dB on PixelNeRF and $+0.15$ dB on GNT) and was therefore adopted for all remaining experiments.

Standardized execution parameters were applied across the benchmark. Camera pose refinement was disabled in Nerfacto because equivalent pose optimization is not natively supported by the other evaluated architectures. Mixed-precision training was enabled for all models, and the hash table capacity in Instant-NGP and Nerfacto was set to 2^{16} with white background compositing. For GNT, structural hyperparameters were kept at 48 coarse and fine samples per ray, a transformer depth of 8, and a feature width of 64 [8].

B. Models & Training Regimes

Five representative architectures were evaluated across two modeling groups: generalizable implicit networks (PixelNeRF [7] and GNT [8]) and per-scene baselines comprising implicit hash grids (Instant-NGP [6] and Nerfacto [3]) and explicit 3D Gaussian primitives (Splatfacto [2]). Three operational regimes were defined: (i) *Zero-Shot (ZS)*, where pre-trained generalizable weights were evaluated directly without gradient updates; (ii) *Fine-Tuning (FT)*, where pre-trained generalizable models were adapted on sparse target views; and (iii) *Per-Scene*, where baseline representations were trained from scratch on $N = 10$ views for $30,000$ iterations.

1) Fine-Tuning Strategies and Implementation: Custom Nerfstudio wrapper packages (`nerfstudio-gnt` and `nerfstudio-pixelnerf`[1]) were implemented with a shared adaptation module (`adaptation.py`) to execute the five strategies described in Section II-C.

During model initialization, `setup_adaptation()` configured the `requires_grad` flags of each sub-network and replaced targeted projection layers. Under *Full-FT*, gradient updates were enabled across both the 2D encoder and the 3D rendering network. Under *PFT-Decoder*, the 2D feature encoder (`feature_net` in GNT and `encoder` in PixelNeRF) was frozen and locked in evaluation mode (`.eval()`), restricting weight updates to the volumetric decoder (`net_coarse`/`net_fine` in GNT and `mlp_coarse`/`mlp_fine` in PixelNeRF). Under *PFT-Backbone*, gradients were enabled only in the 2D encoder while all volumetric rendering layers remained frozen. For *BitFit*, multidimensional weight tensors were frozen and gradients were restricted to 1D bias vectors (`name.endswith(".bias")`), corresponding to less than 0.15% of the model parameters. For *PEFT-LoRA*, the base model was frozen and linear projections (`q_proj`, `v_proj`, and MLP layers) were replaced with `LoRALinear` modules containing trainable rank-r matrices $A \in \mathbb{R}^{r \times d_{\text{in}}}$ (Kaiming-uniform initialization) and $B \in \mathbb{R}^{d_{\text{out}} \times r}$ (zero initialization), scaled by α/r.

Because Nerfstudio registers separate optimizer parameter groups for the 2D encoder and 3D renderer, freezing an entire sub-network produces an empty parameter list in PyTorch. This issue was handled by inserting an inert scalar tensor (`dummy_param` with `requires_grad=False`) into empty groups and overriding `model.train()` to prevent BatchNorm statistics from updating inside frozen encoders. Before test-view evaluation, `merge_lora_weights()` folded the low-rank increment $\Delta W = \frac{\alpha}{r} BA$ directly into the base weight matrix W_0 in-place, avoiding additional matrix multiplications during inference.

[1]Repositories and packages: https://github.com/Felipe-gsilva/Few-Shot-NeRFs/ and https://github.com/Felipe-gsilva/nerf-ann-paper/.

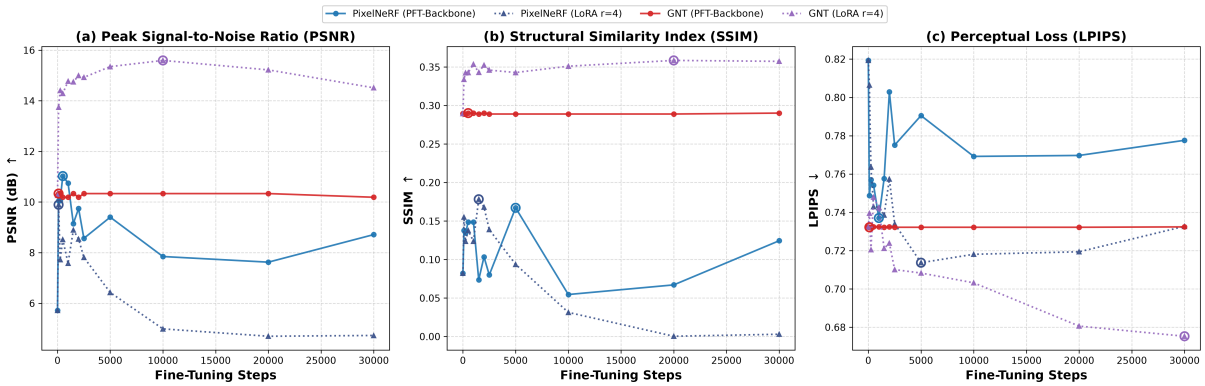


Fig. 2. Ablation of fine-tuning iterations from 0 (Zero-Shot) to $30,000$ steps on the *fern* scene ($N = 3$) for (a) PSNR, (b) SSIM, and (c) LPIPS; circled points mark global optima. PixelNeRF reached its peak between 500 and $1,000$ steps before degrading due to overfitting (4.70 dB under LoRA at 20k steps), whereas GNT with PEFT-LoRA ($r = 4$) obtained $> 91\%$ of its maximum PSNR gain by 500 steps (14.29 vs. 15.59 dB at 10k) and remained stable up to 30k steps.

2) Stratified K-Fold Grid Search and Convergence Calibration: Hyperparameter sensitivity and variance across seeds were evaluated using a Stratified K-Fold protocol ($K = 4$) over all eight LLFF scenes. Each fold paired a foliage scene with a structured indoor or architectural scene: Fold 0 (*fern*, *fortress*), Fold 1 (*flower*, *horns*), Fold 2 (*leaves*, *room*), and Fold 3 (*orchids*, *trex*). The search grid covered learning rates $\eta \in \{10^{-4}, 5 \times 10^{-4}, 10^{-3}\}$ and LoRA ranks $r \in \{4, 8\}$ ($\alpha = 2r$) across three random seeds $\{42, 123, 999\}$ under $N = 3$ input views, optimized with Adam (512 training rays and 128 evaluation rays per batch). Optimization runs that diverged on dense foliage scenes (producing 0.0 dB PSNR on *leaves* and *orchids*) were excluded from the aggregated averages.

The number of fine-tuning iterations for the grid search was selected through an ablation experiment from 0 to $30,000$ steps ($\{0, 100, 250, 500, 1\text{k}, 1.5\text{k}, 2\text{k}, 2.5\text{k}, 5\text{k}, 10\text{k}, 20\text{k}, 30\text{k}\}$) on the *fern* scene ($N = 3$), shown in Figure 2. Four distinct convergence trajectories were observed: (i) *PixelNeRF (PFT-Backbone, $\eta = 10^{-4}$)* increased from 5.72 dB (ZS) to its peak between 500 and $1,000$ steps (11.03–11.53 dB PSNR), after which continued training caused overfitting and reduced PSNR to 7.85 dB at 10k and 7.63 dB at 20k steps; (ii) *PixelNeRF (PEFT-LoRA, $r = 4, \eta = 10^{-3}$)* reached its highest PSNR at 100 steps (9.89 dB) and its lowest LPIPS at $5,000$ steps (0.714), followed by representation degradation beyond $10,000$ steps (4.70 dB PSNR and 0.0004 SSIM at 20k); (iii) *GNT (PFT-Backbone, $\eta = 10^{-4}$)* remained nearly constant ($\approx$ 10.19–10.33 dB) across all $30,000$ steps, indicating that updating the 2D patch embedder alone did not alter epipolar ray aggregation; and (iv) *GNT (PEFT-LoRA, $r = 4, \eta = 10^{-3}$)* reached 14.29 dB by 500 steps ($\sim 92\%$ of its 15.59 dB PSNR peak at $10,000$ steps) and continued improving structural similarity and LPIPS up to 20k–30k steps (0.359 SSIM and 0.675 LPIPS), with a slight PSNR drop at 30k (14.51 dB). Based on these curves, 500 steps was adopted as the standard iteration budget for the K-Fold grid search, avoiding overfitting in PixelNeRF while retaining over 90% of GNT's peak gain within ~ 50 s.

TABLE II
CONSOLIDATED BENCHMARK RESULTS ON THE LLFF DATASET. VALUES REPRESENT GLOBAL AVERAGES ACROSS VALID TEST SCENES. TRAINING TIME CORRESPONDS TO THE BEST 500-STEP FINE-TUNING CONFIGURATION FOR GENERALIZABLE MODELS (INCLUDING TEST-VIEW EVALUATION) AND 30,000-STEP CONVERGENCE FOR PER-SCENE MODELS ($N = 10$). VRAM INDICATES PEAK DYNAMIC MEMORY ALLOCATED DURING INFERENCE.

Model	Regime	PSNR ↑	SSIM ↑	LPIPS ↓	Time ↓	FPS ↑	VRAM ↓
PixelNeRF	ZS	6.35	0.085	0.800	∼22 s	0.004	1.93
	FT	10.92	0.162	0.772	4 min 36 s	0.006	1.97
GNT	ZS	10.51	0.314	0.742	**∼9 s**	0.014	1.94
	FT	15.07	0.368	0.679	4 min 59 s	0.015	1.93
Instant-NGP	N=10	14.06	0.442	0.498	31 min 51 s	0.199	1.09
Nerfacto	N=10	**27.11**	**0.875**	**0.171**	8 min 57 s	0.388	1.43
Splatfacto	N=10	24.80	0.830	0.210	16 min 23 s	**200.7**	**0.10**

C. Evaluation & Metrics

All evaluations were executed on the held-out test views (`llffhold=8`) inside isolated NVIDIA Docker containers using an NVIDIA GeForce GTX 1660 Super GPU (6GB VRAM, CUDA 12.6). Reconstruction quality was measured using PSNR, SSIM [10], and VGG-based LPIPS [5], [11]. Computational performance was recorded through total training/adaptation time, rendering throughput in frames per second (FPS) [9], and peak dynamic VRAM usage allocated by PyTorch during test-view rendering.

IV. RESULTS AND DISCUSSION

This section presents the quantitative, computational, and qualitative results obtained across the LLFF benchmark. Table II reports the global averages for reconstruction fidelity, training time, rendering throughput, and dynamic memory allocation across all evaluated regimes.

A. Visual Fidelity Across Paradigms

Under $N = 10$ input views, per-scene methods that incorporate spatial proposal sampling or explicit 3D primitives

TABLE III

MULTI-SEED EVALUATION OF FINE-TUNING STRATEGIES ON PIXELNERF (5 VALID SCENES) AND GNT (6 VALID SCENES) ACROSS SEEDS $\{42, 123, 999\}$ AT 500 STEPS. CNN-BASED PIXELNERF ACHIEVED ITS HIGHEST PSNR UNDER ENCODER ADAPTATION (*PFT-Backbone*), WHEREAS TRANSFORMER-BASED GNT PERFORMED BEST UNDER EPIPOLAR DECODER TUNING (*PFT-Decoder*) AND LOW-RANK ADAPTATION (*PEFT-LoRA*).

Model	Strategy	η	r	PSNR (dB) ↑	SSIM ↑	LPIPS ↓
PixelNeRF	Zero-Shot (ZS)	N/A	N/A	6.35 ± 0.05	0.085	0.800
	Full-FT	10^{-4}	N/A	9.03 ± 0.11	0.136	0.752
	PFT-Decoder	10^{-4}	N/A	10.58 ± 0.08	**0.205**	0.773
	PFT-Backbone	10^{-4}	N/A	**10.92** ± 0.26	0.162	0.772
	BitFit	10^{-4}	N/A	10.79 ± 0.19	0.119	0.775
	PEFT-LoRA	10^{-3}	4	8.59 ± 0.10	0.175	**0.725**
GNT	Zero-Shot (ZS)	N/A	N/A	10.51 ± 0.11	0.314	0.742
	Full-FT	10^{-3}	N/A	15.06 ± 0.03	**0.369**	0.681
	PFT-Decoder	10^{-3}	N/A	**15.07** ± 0.03	0.368	**0.679**
	PFT-Backbone	10^{-4}	N/A	10.51 ± 0.11	0.314	0.742
	BitFit	10^{-3}	N/A	13.12 ± 0.05	0.358	0.742
	PEFT-LoRA	10^{-3}	4	14.57 ± 0.07	**0.369**	0.706

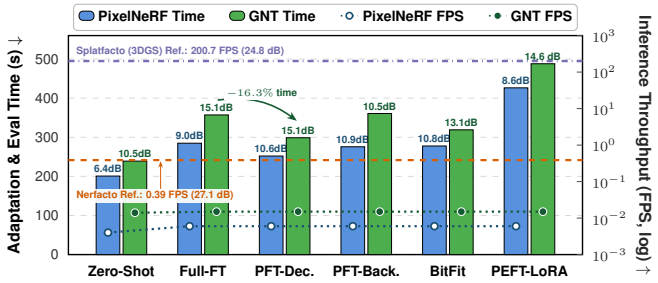


Fig. 3. Adaptation and evaluation time (bars, left axis, annotated with mean PSNR) and rendering throughput in FPS (markers, right log-axis) across fine-tuning strategies for PixelNeRF and GNT, compared against Nerfacto and Splatfacto (3DGS) reference lines. In GNT, *PFT-Decoder* matched *Full-FT* accuracy (15.07 dB) while reducing adaptation time by 16.3% (298.8 s vs. 357.1 s). *PEFT-LoRA* ($r = 4$) required longer training time (488.0 s) due to adapter projections during backpropagation, but maintained native inference speed (0.015 FPS) after in-place weight merging.

achieved the highest visual quality. Nerfacto obtained the best overall metrics at 27.11 dB PSNR, 0.875 SSIM, and 0.171 LPIPS (Table II), followed by Splatfacto (3DGS) with 24.80 dB PSNR, 0.830 SSIM, and 0.210 LPIPS. Initializing anisotropic 3D Gaussians from sparse COLMAP point clouds preserved structural boundaries and high-frequency textures across test views. By comparison, Instant-NGP lacked sparse-view depth regularization and overfitted to the training rays, reaching 14.06 dB PSNR.

For the generalizable architectures, direct Zero-Shot (ZS) inference was affected by domain shift and wide baselines between cameras. PixelNeRF obtained 6.35 dB PSNR and 0.800 LPIPS in zero-shot mode, whereas GNT reached 10.51 dB PSNR and 0.314 SSIM through its epipolar cross-attention mechanism. After fine-tuning on the target views, GNT improved to 15.07 dB PSNR (and up to 17.46 dB on *fortress*), outperforming per-scene Instant-NGP (14.06 dB) in less training time (4 min 59 s vs. 31 min 51 s). Fine-tuning also raised PixelNeRF by +4.57 dB to 10.92 dB PSNR under backbone adaptation, although its rendered views remained spatially blurred.

B. Comparative Analysis of Fine-Tuning Strategies

Table III and Figure 3 summarize the multi-seed reconstruction metrics and execution times across the five fine-tuning strategies for PixelNeRF (5 valid scenes) and GNT (6 valid scenes). The results show opposite adaptation requirements between the convolutional and transformer-based architectures.

In PixelNeRF, the highest PSNR was obtained when restricting gradient updates to the ResNet encoder (*PFT-Backbone*, 10.92 ± 0.26 dB in 275.7 s) or to additive biases (*BitFit*, 10.79±0.19 dB in 277.7 s), both of which outperformed *Full-FT* (9.03 ± 0.11 dB in 284.8 s). Because PixelNeRF projects 2D feature maps onto 3D rays, adjusting the convolutional filters to the target scene illumination improved color consistency, whereas updating all weights simultaneously destabilized the downstream MLP. Table III also indicates a difference between pixel-level and perceptual metrics: *PFT-Backbone* maximized PSNR through smooth color averaging, while *PFT-Decoder* reached the highest structural similarity (0.205 SSIM in 252.0 s) and *PEFT-LoRA* ($r = 4$) obtained the lowest perceptual error (0.725 LPIPS vs. 0.772 for *PFT-Backbone*) despite a lower PSNR (8.59 dB).

In GNT, the best reconstruction quality was achieved under *PFT-Decoder* (15.07 ± 0.03 dB PSNR and 0.679 LPIPS), matching *Full-FT* (15.06 ± 0.03 dB) while avoiding back-propagation through the ViT encoder and reducing adaptation time from 357.1 s to 298.8 s (a 16.3% reduction, Figure 3). Freezing the pre-trained ViT encoder preserved its visual representations and concentrated weight updates within the epipolar cross-attention layers. In contrast, *PFT-Backbone* produced no improvement over Zero-Shot (10.51±0.11 dB in 360.8 s), showing that GNT adaptation depends on epipolar ray aggregation rather than shallow 2D patch projections. Parameter-efficient strategies also regularized GNT training: *BitFit* reached 13.12±0.05 dB (+2.61 dB over ZS in 319.0 s) by updating $< 0.15\%$ of the weights, and *PEFT-LoRA* ($r = 4$) reached 14.57 ± 0.07 dB and 0.369 SSIM. Although evaluating the auxiliary `LoRALinear` branches during training increased execution time (488.0 s in GNT and 426.4 s in PixelNeRF), merging $\Delta W = \frac{\alpha}{r} BA$ into W_0 before evaluation preserved the baseline rendering speed (0.015 and 0.006 FPS).

C. Computational Efficiency and Pareto Trade-offs

Figures 4 and 5 present the PSNR distribution across scenes and the five-axis trade-off profile across fidelity and hardware metrics. Generalizable models exhibited low inter-scene variance in Zero-Shot mode, which expanded after fine-tuning as the networks adapted to scene-specific geometry, whereas per-scene models showed wider variance driven by scene texture complexity.

Comparing training time against inference throughput highlights the operational trade-off between both families of methods. Generalizable models required only ∼ 9–22 s for Zero-

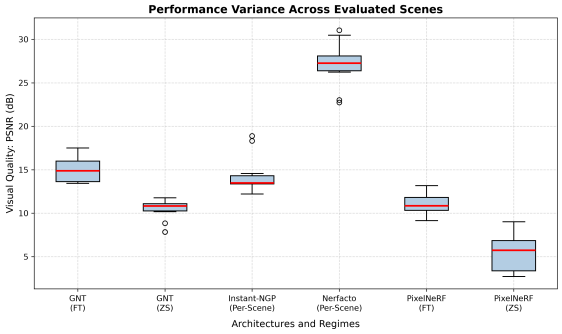


Fig. 4. PSNR distributions across valid LLFF scenes ($N = 10$ for per-scene baselines; optimal FT vs. ZS for generalizable models). Red horizontal lines indicate medians and boxes represent interquartile ranges (IQR). Fine-tuned GNT achieved a higher median PSNR than Instant-NGP (15.07 vs. 14.06 dB), while Nerfacto reached the highest overall distribution (27.11 dB mean).

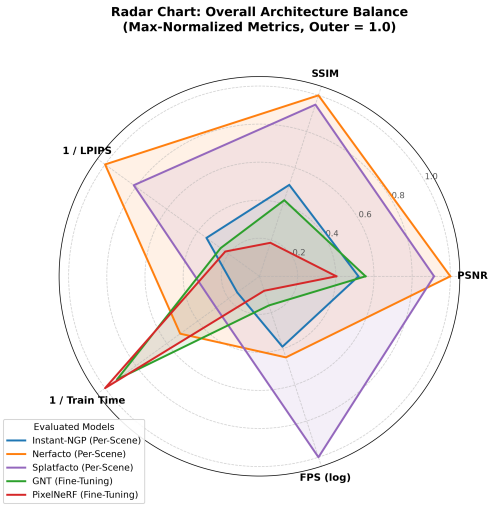


Fig. 5. Multi-metric trade-offs under sparse-view conditions ($N = 10$). Metrics were max-normalized to $[0, 1]$ (outer perimeter = best), using $1/\text{LPIPS}$, $1/\text{Train Time}$, and log-scaled FPS. Splatfacto led in rendering speed (200.7 FPS) and Nerfacto led in visual fidelity, whereas fine-tuned GNT required the shortest adaptation time (~ 5 min) but exhibited low rendering FPS.

Shot evaluation or $\sim$ 4.6–5.0 min for 500-step fine-tuning, but projecting multi-view features and evaluating epipolar attention along each ray limited rendering throughput to 0.006 FPS in PixelNeRF and 0.015 FPS in GNT, with $\sim$ 1.94 GB of dynamic VRAM. Per-scene hash-grid models increased inference speed to 0.199 FPS in Instant-NGP and 0.388 FPS in Nerfacto. Using explicit tile-based rasterization, Splatfacto (3DGS) reached 200.7 FPS ($\approx 517\times$ faster than Nerfacto) while consuming 0.10 GB of dynamic VRAM ($14.3\times$ less than Nerfacto's 1.43 GB), requiring 16 min 23 s of per-scene optimization. Consequently, PFT- and PEFT-adapted GNT was advantageous under extreme view scarcity ($N = 3$) and short adaptation budgets, whereas Splatfacto and Nerfacto were superior when $N \geq 10$ views and longer training times were available.

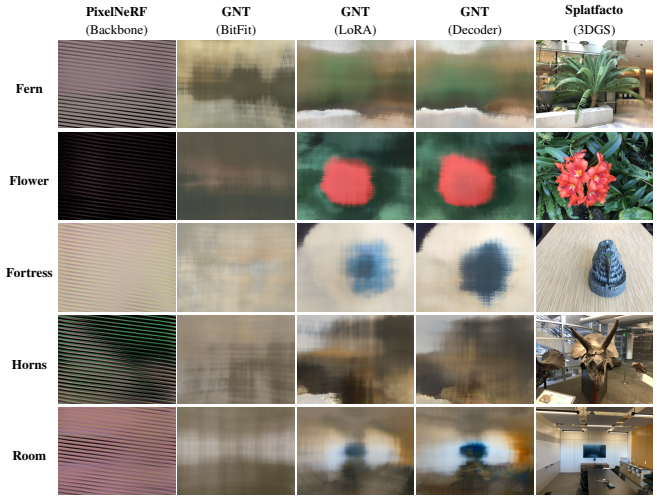


Fig. 6. Qualitative comparison across five LLFF scenes under $N = 10$ input views. From left to right: PixelNeRF adapted with PFT-Backbone (Col. 1), GNT adapted with BitFit (Col. 2), PEFT-LoRA ($r = 4$, Col. 3), and PFT-Decoder (Col. 4), and per-scene Splatfacto / 3DGS (Col. 5). Increasing the adapted parameter capacity in the GNT decoder progressively sharpened object contours, while Splatfacto preserved the finest high-frequency details at > 200 FPS.

D. Qualitative Analysis

Figure 6 shows novel views rendered across five LLFF scenes (*Fern*, *Flower*, *Fortress*, *Horns*, and *Room*) under $N = 10$ input views.

In Column 1, PixelNeRF adapted with *PFT-Backbone* recovered the global color palette and coarse object contours in structured scenes such as *Fortress* and *Room*. However, because its coordinate MLP averages projected features without explicit depth reasoning, scenes with multi-layer occlusions (*Fern* and *Horns*) exhibited heavy spatial smoothing, consistent with its 0.772 LPIPS score.

Columns 2 to 4 show the visual progression across the GNT fine-tuning strategies as the number of adapted parameters in the decoder increased. In Column 2, *BitFit* ($< 0.15\%$ trainable weights) corrected zero-shot color shifts and reduced near-camera floaters, but left fine surface textures blurred. In Column 3, adding rank-4 adapters via *PEFT-LoRA* sharpened object boundaries, recovering petal edges in *Flower* and bone contours in *Horns*. In Column 4, updating the full rendering head under *PFT-Decoder* produced the sharpest reconstructions among the generalizable models (15.07 dB PSNR and 0.679 LPIPS), separating overlapping leaves in *Fern* and stone reliefs in *Fortress*.

In Column 5, *Splatfacto* (3DGS) produced the sharpest textures and depth boundaries across all five scenes. The explicit 3D Gaussian primitives reconstructed fine foliage details in *Fern* and *Flower* as well as background structures in *Horns* and *Room* at 200.7 FPS, showing only localized elongated Gaussian artifacts near image borders where camera overlap was lower.

V. Conclusion

This work presented a reproducible benchmarking methodology to evaluate reconstruction fidelity, adaptation stability, and computational efficiency across generalizable and per-scene neural rendering architectures under sparse-view conditions. In the experimental evaluation, per-scene models achieved the best visual quality and rendering speed under $N = 10$ views: Nerfacto reached 27.11 dB PSNR and 0.171 LPIPS, while Splatfacto (3DGS) rendered novel views at 200.7 FPS using 0.10 GB of dynamic VRAM. Under three input views ($N = 3$), modular fine-tuning of pre-trained priors prevented representational collapse: adapting GNT via *PFT-Decoder* (15.07 dB) or *PEFT-LoRA* (14.57 dB) reduced adaptation time by 16.3% compared to full fine-tuning and outperformed Instant-NGP (14.06 dB), although limited by volumetric ray-marching throughput (≤ 0.015 FPS).

Future work will extend this evaluation protocol to unbounded 360° scenes and investigate depth-regularized sparse-view 3D Gaussian Splatting methods [13], [21] alongside feed-forward Gaussian architectures.

Acknowledgment

The authors gratefully acknowledge the financial support of the National Council for Scientific and Technological Development - CNPq (Grants #305386/2024-7, #302833/2025-0, #307318/2022-2 and #404571/2025-5), the State of Minas Gerais Research Foundation - FAPEMIG (Grant #APQ-00727-24 and #APD-01428-25) and the State of São Paulo Research Foundation - FAPESP (Grant #2022/03020-1).

References

[1] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "Nerf: Representing scenes as neural radiance fields for view synthesis," 2020.

[2] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3d gaussian splatting for real-time radiance field rendering," *ACM Transactions on Graphics*, vol. 42, no. 4, July 2023.

[3] M. Tancik, E. Weber, E. Ng, R. Li, B. Yi, T. Wang, A. Kristoffersen, J. Austin, K. Salahi, A. Ahuja, D. Mcallister, J. Kerr, and A. Kanazawa, "Nerfstudio: A modular framework for neural radiance field development," in *Special Interest Group on Computer Graphics and Interactive Techniques Conference Conference Proceedings*, ser. SIGGRAPH '23. ACM, Jul. 2023, p. 1–12.

[4] A. Laurino, A. Franceschini, L. Pesce, L. Cinci, A. Montalbano, G. Mazzamuto, G. Sancataldo, G. Nesi, I. Costantini, L. Silvestri, and F. S. Pavone, "A guide to perform 3d histology of biological tissues with fluorescence microscopy," *International Journal of Molecular Sciences*, vol. 24, no. 7, 2023.

[5] M. Niemeyer, J. T. Barron, B. Mildenhall, M. S. Sajjadi, A. Geiger, and N. Radwan, "Regnerf: Regularizing neural radiance fields for view synthesis from sparse inputs," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 5480–5490.

[6] T. Müller, A. Evans, C. Schied, and A. Keller, "Instant neural graphics primitives with a multiresolution hash encoding," *ACM Transactions on Graphics*, vol. 41, no. 4, p. 1–15, Jul. 2022.

[7] A. Yu, V. Ye, M. Tancik, and A. Kanazawa, "pixelnerf: Neural radiance fields from one or few images," 2021.

[8] Z. Wang, M. Varma, P. Wang, X. Chen, T. Chen, and S. Venugopalan, "Is attention all that nerf needs?" 2023.

[9] J. Kulhanek and T. Sattler, "Nerfbaselines: Consistent and reproducible evaluation of novel view synthesis methods," 2025.

[10] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: from error visibility to structural similarity," *IEEE transactions on image processing*, vol. 13, no. 4, pp. 600–612, 2004.

[11] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, "The unreasonable effectiveness of deep features as a perceptual metric," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 586–595.

[12] A. Jain, M. Tancik, and P. Abbeel, "Putting NeRF on a diet: Semantically consistent few-shot view synthesis," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021, pp. 5885–5894.

[13] K. Seo, S. Hyun, M. Lee, and J.-P. Heo, "Improving sparse-view 3dgs generalization via flat minima optimization," 2026.

[14] A. Chen, Z. Xu, F. Zhao, X. Zhang, F. Xiang, J. Yu, and H. Su, "Mvsnerf: Fast generalizable radiance field reconstruction from multi-view stereo," 2021.

[15] Q. Wang, Z. Wang, K. Genova, P. P. Srinivasan, H. Zhou, J. T. Barron, R. Martin-Brualla, N. Snavely, and T. Funkhouser, "IBRNet: Learning multi-view image-based rendering," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 4690–4699.

[16] B. Mildenhall, P. P. Srinivasan, R. Ortiz-Cayon, N. K. Kalantari, R. Ramamoorthi, R. Ng, and A. Kar, "Local light field fusion: Practical view synthesis with prescriptive sampling guidelines," 2019.

[17] B. Roessle, J. T. Barron, B. Mildenhall, P. P. Srinivasan, and M. Nießner, "Dense depth priors for neural radiance fields from sparse input views," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 12 892–12 901.

[18] D. Gupta, A. Bhatti, S. Parmar, C. Dan, Y. Liu, B. Shen, and S. Lee, "Beyond LoRA: Exploring efficient fine-tuning techniques for time series foundational models," *arXiv preprint arXiv:2409.11302*, 2024.

[19] E. Ben Zaken, S. Ravfogel, and Y. Goldberg, "BitFit: Simple parameter-efficient fine-tuning for transformer-based masked language-models," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022, pp. 1–9.

[20] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations (ICLR)*, 2022.

[21] M. Song, X. Lin, D. Zhang, H. Li, X. Li, B. Du, and L. Qi, "D^2gs: Depth-and-density guided gaussian splatting for stable and accurate sparse-view reconstruction," 2025.